# The Architecture of Linux CPU Scheduling: CFS, EEVDF, and Throughput Limits for High-Concurrency PostgreSQL Workloads

The fundamental imperative of any general-purpose operating system scheduler is to answer a single question thousands of times per second: which task should occupy the physical CPU next? This decision algorithm must constantly balance competing and often mutually exclusive priorities. It must enforce fairness to ensure no process starves, maintain high execution speed to ensure the selection mechanism consumes minimal cycles, deliver extreme responsiveness to guarantee low latency for interactive or real-time tasks, and exhibit robust scalability to operate efficiently across hundreds of modern multi-core processors[1].

For nearly two decades, the Linux kernel relied on the Completely Fair Scheduler (CFS), a mathematically elegant mechanism introduced in version 2.6.23 in October 2007[1]. CFS represented a radical paradigm shift away from the fragile heuristics of the previous $O(1)$ scheduler, prioritizing strict proportional fairness above all else[2]. However, as datacenter workloads evolved and the demand for microsecond-scale tail-latency reduction intensified, the limitations of the CFS architecture became acutely apparent. Optimized primarily for equitable long-term throughput, CFS lacked a native mechanism to guarantee deterministic latency for urgent tasks without relying on external priority escalation or out-of-band hacks[1].

In October 2023, with the release of Linux kernel 6.6, the mainline kernel formally transitioned its default SCHED_NORMAL scheduling class to the Earliest Eligible Virtual Deadline First (EEVDF) algorithm[5]. Based on foundational research published in 1995 by Ion Stoica and Hussein Abdel-Wahab, EEVDF introduces a rigorous mechanism to explicitly enforce time deadlines while maintaining the proportional-share fairness guarantees established by CFS[4]. This architectural shift, subsequently combined with major kernel preemption model modifications introduced in Linux 7.0[7], has profound implications for high-throughput, process-heavy workloads. This is critically true for monolithic relational databases like PostgreSQL, which rely on rigid process-per-connection architectures and user-space synchronization primitives[8].

This report provides an exhaustive technical analysis of the internal mechanics of both the CFS and EEVDF schedulers, detailing their mathematical foundations, source code implementations within kernel/sched/fair.c, scheduling periodicities, and task processing capacities. Building upon this operating system foundation, the analysis culminates in a rigorous mathematical deduction regarding the absolute physical limits of active PostgreSQL connections that can be sustained on a single CPU core before context-switch overhead induces systemic throughput collapse.

# The Completely Fair Scheduler (CFS) Architecture

To fully comprehend the structural advantages of EEVDF, one must first analyze the internal mechanics of the architecture it replaced. CFS abandoned traditional time-slicing and complex interactivity heuristics, aiming instead to model a theoretical "ideal multi-tasking CPU"[2]. In this idealized hardware state, every runnable task in the system would execute simultaneously in parallel, receiving a mathematically precise fraction of processor resources directly proportional to its assigned priority weight[3].

Because true simultaneous execution of multiple tasks on a single physical core is impossible, CFS approximated this ideal state by tracking the normalized progress of each individual task using a foundational metric known as virtual runtime, or vruntime[3].

## The Mathematics of vruntime and Task Weight

The vruntime of a scheduling entity represents its actual physical execution time on the CPU, scaled inversely by its relative priority weight[3]. A task's weight is derived from its user-space nice value. The core update mechanism embedded within the kernel increments a task's vruntime based on the delta of its physical execution time across each scheduler tick[4].

The mathematical formulation for the load weight $\lambda_i$ associated with a process is an exponential function of its nice value $\nu_i$, such that $\lambda_i \simeq 1.25^{-\nu_i}$. For computational efficiency within the kernel, this is scaled by a factor of $2^{10}$, resulting in the physical weight $w_i$ [4]. When a task executes for a physical time delta $\Delta t$, its virtual runtime is incremented via the following relationship:

$$vruntime_i += \Delta t \times \frac{W_{NICE\_0}}{w_i}$$

where $W_{NICE\_0}$ represents the default weight of a process with a neutral nice value of 0 (conventionally defined as 1024), and $w_i$ represents the specific weight of task $i$[4]. In a perfectly balanced theoretical scenario, all tasks residing on the runqueue would maintain the exact same vruntime value, indicating perfect fairness and synchronous progression[3].

## Data Structures: The Red-Black Tree and min_vruntime

Within the kernel source code, CFS maintained runnable tasks inside a time-ordered red-black tree (RB-tree) data structure attached to the central runqueue struct cfs_rq[1]. The scheduler's core selection logic, housed in the pick_next_task_fair function, was remarkably simple: it consistently selected the "leftmost" node in the RB-tree, corresponding directly to the task possessing the absolute smallest vruntime—the entity that had been mathematically deprived of the most CPU time relative to its peers[3].

To preserve the integrity of the tree and prevent newly awakened tasks (which might have

been sleeping for days and thus possess a vruntime near zero) from starving all other tasks, CFS maintained a critical runqueue-level variable called cfs_rq->min_vruntime[3]. This metric tracked a monotonically increasing value representing the smallest vruntime currently active among all tasks in the runqueue[3]. When a sleeping task woke up, its vruntime was artificially advanced to a value slightly ahead of min_vruntime, ensuring it received prompt attention without monopolizing the CPU entirely.

| CFS Property | Implementation Detail | Operational Implication |
|---|---|---|
| **Selection Criterion** | Minimum vruntime | Always selects the task mathematically owed the most CPU time[1]. |
| **Data Structure** | Red-Black Tree sorted by vruntime | $O(\log N)$ insertion, but effectively $O(1)$ selection by caching the leftmost node pointer[3]. |
| **Time Slice** | Dynamic and Implicit | Slices are computed relative to total runqueue weight rather than strict absolute time[1]. |
| **Fairness Guarantee** | vruntime convergence | Excellent long-term proportional equity across batch workloads[1]. |

## Scheduling Periodicities and Task Processing Capacities in CFS

A critical aspect of any scheduler is how often it evaluates tasks and how many it can physically process within a single scheduling period or "epoch." CFS did not rely on fixed, static time slices like older algorithms. Instead, it operated over a dynamic target schedule latency.

The primary tunable parameter was sched_latency_ns (historically defaulting to 6,000,000 nanoseconds, or 6ms)[4]. This value represented the targeted time frame in which every runnable process on the queue should ideally be given at least one chance to execute on the CPU[4]. To prevent excessive context-switching under heavy load, CFS paired this with a sched_min_granularity_ns threshold (typically defaulting to 0.75ms)[4].

The physical capacity of tasks processed per scheduling period on a single core was mathematically defined by these two boundaries. If the system targeted a 6ms epoch and enforced a 0.75ms minimum execution slice per task to preserve cache locality, the maximum number of tasks that could be optimally serviced in a single scheduling period was precisely

calculated as:

$$\text{Maximum Tasks per Epoch} = \frac{\text{sched\_latency\_ns}}{\text{sched\_min\_granularity\_ns}} = \frac{6.0\text{ms}}{0.75\text{ms}} = 8 \text{ tasks}$$

If the runqueue expanded beyond 8 active tasks, CFS recognized that dividing 6ms among more than 8 entities would violate the 0.75ms minimum granularity rule, leading to unacceptable context switch thrashing[13]. Consequently, the scheduler would dynamically stretch the epoch length. For example, with 20 active tasks, the scheduling period would expand to $20 \times 0.75\text{ms} = 15\text{ms}$. Therefore, while CFS could technically process an unbounded number of tasks, it was structurally optimized to cycle through approximately 8 tasks per period on a single core under default tuning[12].

### The Latency vs. Fairness Dichotomy

CFS's fatal flaw lay in its exclusive optimization for "longest-averaged-time-to-finish." It possessed no native vocabulary for task urgency[1]. If a highly interactive, latency-sensitive task woke up after a brief sleep, it was assigned a vruntime near the current min_vruntime. If several heavy CPU-bound tasks also happened to possess vruntime values clustered tightly near that minimum, the waking task was forced into an arbitrary queue position[1].

The latency-sensitive task might be forced to wait for multiple heavy tasks to exhaust their full scheduling quantums before it could finally acquire the CPU. This resulted in observable micro-stutters in desktop environments and unacceptable tail latencies for high-frequency network servers, driving the kernel community to seek a scheduler capable of natively prioritizing latency without compromising mathematically proven fairness[1].

## The Earliest Eligible Virtual Deadline First (EEVDF) Algorithm

EEVDF resolves the fundamental latency-fairness dichotomy by structurally disentangling the concepts of resource entitlement (fairness) and time urgency (latency)[5]. It accomplishes this by tracking a task's historical service "lag" and executing scheduling selections based strictly on virtual deadlines[5].

Originally conceptualized in research by Stoica and Abdel-Wahab in 1995, EEVDF operates as a dynamic priority scheduling algorithm designed specifically for proportional share resource allocation[5]. Its primary objective is to achieve bandwidth preservation with deviations bounded explicitly by the size of the time quantum, thereby guaranteeing that no task receives significantly more or less than its entitled share over extended periods, while simultaneously guaranteeing maximum latency boundaries[5].

### Core Mechanics: Lag and the Eligibility Criterion

The foundational metric distinguishing EEVDF from CFS is the concept of "lag." Lag mathematically quantifies the service debt or service surplus a task holds relative to its

idealized, perfectly fair share of CPU time[5]. If a task has received less CPU time than it is rightfully entitled to based on its weight, it holds a positive lag (the system owes the task time). Conversely, if a task has aggressively consumed more than its fair share, it holds a negative lag[6].

The actual lag of process $p$ at physical time $t$, relative to the global virtual time of the runqueue $\bar{\rho}(t)$, is defined in the kernel as:

$$lag_p(t) = w_p(\bar{\rho}(t) - \rho_p(t))$$

where $\rho_p(t)$ is the specific vruntime of the task[4].
Before a task can even be considered for execution by the CPU, it must pass an eligibility check. A task is considered "eligible" to compete for the CPU solely if its lag is greater than or equal to zero[4]. Within the modern Linux implementation in kernel/sched/fair.c, the function vruntime_eligible executes this check by directly comparing the task's vruntime against the runqueue's average vruntime (cfs_rq->avg_vruntime), adjusting for weight and precision loss constraints[17]. If a task's $vruntime \leq avg\_vruntime$, it is flagged as eligible[4]. This mechanism mathematically guarantees fairness, as no task can dominate the processor if it is running ahead of its scheduled entitlement[4].

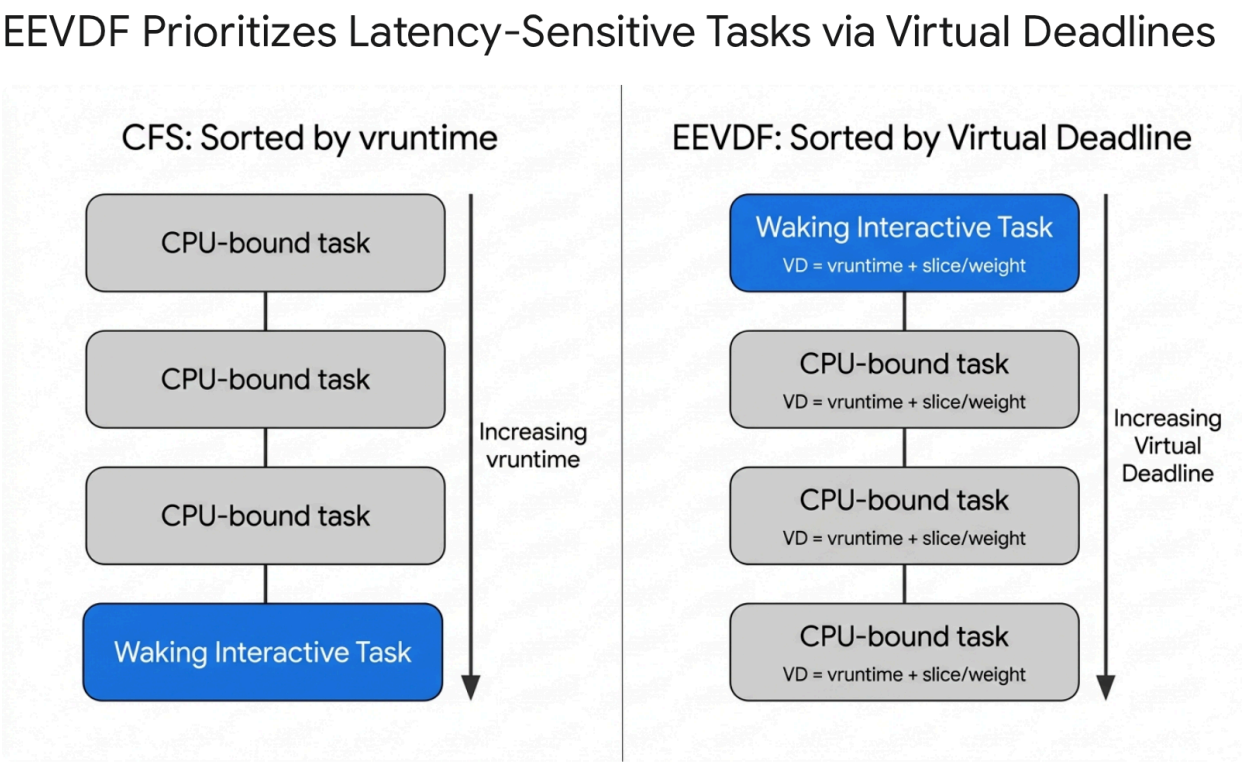
## Virtual Deadlines: Encoding Urgency

Once the kernel establishes the set of all eligible tasks, EEVDF must execute a decision on which specific task to run. It does so by calculating a Virtual Deadline ($VD$) for each eligible entity[6]. The virtual deadline represents the absolute point in virtual time by which the task must complete its requested physical time slice to maintain perfect systemic fairness[5].
The deadline is calculated via the formula:

$$VD_i = V(e_i) + \frac{r_i}{w_i}$$

where $V(e_i)$ represents the virtual eligible time (essentially the current virtual time when the request was initiated), $r_i$ represents the requested time slice length, and $w_i$ is the task's priority weight[4].
The core selection logic of EEVDF is absolute: it scans the pool of eligible tasks and selects the entity with the **earliest virtual deadline**[1]. Because a shorter requested time slice ($r_i$) mathematically results in a proportionally smaller $VD_i$, tasks that request very short bursts of CPU time—such as latency-sensitive network servers or audio processing threads—are

organically prioritized by the algorithm[4]. They jump to the front of the execution queue without requiring any external priority inflation or opaque heuristics[15].




Unlike CFS, which strictly orders tasks by accumulated virtual runtime, EEVDF evaluates eligibility (lag ≥ 0) and then sorts by virtual deadline. A newly awakened latency-sensitive task with a short time slice receives an earlier deadline, allowing it to preempt longer-running tasks without violating overall proportional fairness.

## EEVDF Kernel Implementation: The Augmented Red-Black Tree

Operating EEVDF in a high-performance kernel runqueue presents a massive algorithmic challenge. The scheduler must dynamically filter thousands of entities for eligibility based on constantly shifting vruntime variables, and subsequently identify the entity possessing the minimum virtual deadline, all while supporting real-time task insertions, deletions, and preemption checks[5].

The Linux implementation resolves this via a sophisticated augmented Red-Black tree structure[5]. In initial prototype patches, the tree was ordered by vruntime (representing service), while simultaneously functioning as a heap for deadlines by maintaining a min_deadline attribute at every individual node[19]. This allowed the pick_eevdf function to perform an EDF-like (Earliest Deadline First) search on the subtrees[20].

However, a critical performance optimization introduced during the finalization of the code (notably in a patch series by Abel Wu and Peter Zijlstra) fundamentally inverted this internal structure[17]. In the production implementation located within kernel/sched/fair.c, the RB-tree is

explicitly **sorted by virtual deadline** rather than vruntime. Concurrently, the tree functions as a heap based on vruntime by tracking the min_vruntime of each subtree[19].
This structural augmentation is paramount because it ensures that the core pick_eevdf() function can avoid doubling the worst-case search cost. By utilizing the cached leftmost node of the deadline-sorted tree, the scheduler enables an $O(1)$ fast-path execution for task selection, drastically reducing the CPU cycles burned during the scheduling routine, while maintaining $O(\log n)$ efficiency for task enqueue/dequeue operations[5].

## Sleeper Fairness, Lag Decay, and Deferred Dequeue

A historic vulnerability in proportional-share schedulers involves a gaming exploit by intermittent tasks. A process could execute heavily, accumulate severe negative lag (indicating it had vastly exceeded its fair share), and then intentionally issue a sleep command. Upon waking, traditional schedulers would often reset the task's parameters, effectively erasing its scheduling penalty[6].
EEVDF systematically neutralizes this exploit by fundamentally altering the lifecycle of sleeping tasks. When a task possessing negative lag goes to sleep, the kernel does not immediately evict it from the runqueue[6]. Instead, the task is marked for **"deferred dequeue"**[6]. The task physically remains on the cfs_rq structure but is flagged as an ineligible entity[18].
As global virtual time progresses via the execution of other tasks, the sleeping task's lag slowly decays over virtual run time (VRT) back toward zero[6]. Once the lag crosses the mathematical threshold into positive territory—meaning the task has finally "paid off" its execution debt while sleeping—the scheduler officially removes it from the runqueue[18]. This architectural mechanism guarantees that tasks engaging in short sleeps cannot escape their negative lag penalty, while genuinely long-sleeping processes will eventually have their accumulated debt forgiven[18].

## Customizable Task Quantums and Scheduling Periods in EEVDF

Unlike CFS, which dynamically calculated time slices entirely internally, EEVDF exposes fine-grained control over tail latencies to the application layer. User-space programs can explicitly request specific desired time slices (in nanoseconds) utilizing the sched_setattr() system call, specifically populating the sched_runtime field within the sched_attr data structure[4].
When tasks do not explicitly request a custom slice, the kernel enforces a default baseline quantum defined by the tunable sysctl_sched_base_slice parameter[12]. In the Linux 6.6+ kernel environment, sched_base_slice defaults to precisely 0.75 milliseconds (750,000 nanoseconds), which is scaled logarithmically based on the number of online CPUs via the SCHED_TUNABLESCALING_LOG setting[10]. The implementation of sched_base_slice completely deprecates and replaces the obsolete sched_min_granularity and sched_wakeup_granularity variables utilized by CFS[21].
Regarding scheduling periods, EEVDF does not strictly adhere to a rigid global epoch like CFS's sched_latency_ns[1]. Instead, the effective scheduling period is an emergent property dictated by the sum of the requested slices among all eligible tasks[22]. If a system contains exactly 8

CPU-bound tasks of equal weight on a single core, and each falls back to the default 0.75ms base slice, the core will process exactly 8 tasks over a continuous 6ms period before rotating back to the first task, ensuring highly predictable periodicity[22]. However, if an application leverages sched_setattr() to request a massive 100ms slice, while another requests a 0.1ms slice, the scheduling boundaries become highly variable, and the short-slice task will preempt the longer task continually to enforce its tighter deadline[22].

# The Physical Constraints of Scheduling: Context Switch Overheads

While algorithmically optimizing time slices yields perfect theoretical fairness, physical hardware imposes strict boundaries on how small a time slice can be before systemic collapse occurs. The efficiency of the CPU scheduler is fundamentally bottlenecked by the physical cost of swapping tasks—the context switch.

Whenever the scheduler preempts one task to execute another, the CPU must jump to kernel mode, flush processor registers, save the state of the outgoing thread, update process metadata, load the page tables of the incoming thread, restore its registers, and execute a jump back to user space[24].

## Direct vs. Indirect Overhead Costs

The overarching cost of a context switch is bifurcated into direct and indirect penalties:

1. **Direct Costs:** This is the raw computational cycle cost of executing the kernel's context switch mechanism. On modern x86 Linux architectures, the direct cost is highly optimized, generally consuming between 1.2 and 1.6 microseconds when the process remains pinned to a single CPU core[25]. If the scheduler decides to migrate the thread across different cores to balance load, this cost inflates to approximately 2.3 microseconds[25].
2. **Indirect Costs:** The indirect penalty is vastly more severe and stems from cache and Translation Lookaside Buffer (TLB) annihilation[24]. When a new process is scheduled onto the CPU, its memory access patterns differ entirely from the previous task. The processor must evict the previous task's working set from the L1 and L2 caches, and flush the TLB[24]. Depending on the dataset size, subsequent cache warm-ups can inflate the true effective cost of a context switch from a few microseconds to well over 200 microseconds as the CPU stalls waiting for main memory (RAM) fetches[27].

If an application is subjected to excessive preemption, generating thousands of context switches per second, the indirect cache destruction results in a scenario where the CPU spends the vast majority of its cycles managing memory stalls rather than advancing program execution[24].

# Kernel Preemption Models and the PREEMPT_LAZY Paradigm

The rate at which these destructive context switches occur is dictated not solely by the EEVDF

quantum, but fundamentally by the kernel's overarching preemption configuration. The kernel preemption model determines *when* the scheduler is legally permitted to interrupt a running user-space thread.

Historically, throughput-oriented servers deployed the PREEMPT_NONE or PREEMPT_VOLUNTARY models[7]. These conservative settings dictated that the kernel would almost never forcibly interrupt a running thread; a process would retain the CPU until it explicitly yielded, executed a blocking system call, or definitively exhausted its entire scheduler time slice[9]. This maximized raw throughput by minimizing context switches and preserving cache locality[7]. Conversely, PREEMPT_FULL allowed the kernel to interrupt threads at almost any safe point to minimize latency, a setting favored by desktop environments[9].

In the Linux 7.0 release, kernel developers executed a massive architectural consolidation by removing the PREEMPT_NONE and PREEMPT_VOLUNTARY models on primary architectures (x86, arm64)[9]. In their place, the kernel defaults to PREEMPT_LAZY[7].

PREEMPT_LAZY operates as a dynamic middle ground. When a higher-priority task wakes up, the kernel does not immediately preempt the current task. Instead, it sets a TIF_NEED_RESCHED_LAZY flag[7]. The preemption is deferred until the running task reaches a natural boundary, exhausts its slice, or until the next periodic scheduler tick arrives[7]. While intended to optimize both latency and throughput, the forced transition to PREEMPT_LAZY introduced catastrophic systemic failures for high-concurrency workloads that rely on user-space synchronization primitives.

## The Lock Convoying Crisis and Page Faults

The dangers of PREEMPT_LAZY become devastatingly apparent in applications that maintain user-space spinlocks. If a thread acquires a user-space spinlock and subsequently triggers a minor page fault (e.g., accessing memory that has been allocated but not yet physically mapped by the OS), the kernel must intervene to handle the fault[8].

Under PREEMPT_LAZY, the scheduler observes the faulting thread stalling and dynamically preempts it, reallocating the CPU to another runnable task[8]. However, the preempted thread remains in possession of the user-space spinlock. As the scheduler cycles through other waiting threads, they attempt to acquire the exact same spinlock. Because the lock holder is asleep in the kernel, the competing threads enter an endless busy-wait state (spinning), violently burning through their entire EEVDF time slices while executing zero productive instructions[9]. This cascading starvation—known as lock convoying—rapidly destroys overall system throughput[8].

## Restartable Sequences (RSEQ) and Time-Slice Extensions

To mitigate the catastrophic impact of preempting tasks inside critical lock-holding sections, Linux kernel developers led by Peter Zijlstra and Thomas Gleixner engineered a sophisticated bypass utilizing the Restartable Sequences (RSEQ) user-space ABI[30]. This feature introduces "scheduler time slice extensions"[30].

Under this paradigm, when a thread prepares to enter a critical section to acquire a user-space spinlock, it sets a REQUEST bit in its designated RSEQ memory structure[31]. If the scheduler

attempts to execute a PREEMPT_LAZY eviction while the thread is active, the kernel intercepts the action by inspecting the REQUEST bit. If set, the kernel clears the REQUEST bit, toggles a GRANTED bit, and explicitly extends the thread's time slice, allowing it to complete the critical section and release the lock uninterrupted[31]. Once the thread exits the critical section, it checks for the GRANTED bit; if true, the application must immediately invoke the sys_rseq_slice_yield() system call to voluntarily surrender the CPU, thereby preventing malicious processes from abusing the extension mechanism to indefinitely monopolize the processor[30].

# PostgreSQL Architectural Constraints and Concurrency Limitations

The intersection of EEVDF deadline mathematics, PREEMPT_LAZY convoying risks, and context switch cache annihilation forms the absolute boundary conditions governing the performance of major relational database systems, most notably PostgreSQL[9].

Unlike modern thread-based databases, PostgreSQL adheres to a rigid, legacy process-per-connection architecture[33]. When a client establishes a connection, the central postmaster daemon forks an entirely new OS-level backend process dedicated exclusively to that connection[33]. Each individual process demands a heavy base memory footprint—typically between 5MB and 20MB—merely to maintain idle state, accommodating local memory contexts, prepared statements, and private work_mem allocations[33].

## The Mechanism of Throughput Collapse

When database administrators naively elevate the max_connections parameter to handle massive client load, they frequently induce a state of "throughput collapse"[28]. If 500 active connections attempt to query the database simultaneously on a constrained hardware profile (e.g., an 8-core server), the operating system must manage an extreme load scenario.

The EEVDF scheduler attempts to equitably divide the limited CPU cores among the 500 active processes. Assigning the default sched_base_slice of 0.75ms to each process triggers high-frequency, aggressive context switching. The consequences are systemic and severe:

1. **Context Switch Saturation:** The processor spends an unacceptable percentage of its physical cycles trapped in kernel mode swapping process page tables rather than executing database application logic[24].
2. **Cache Annihilation:** Because each discrete PostgreSQL process operates on unique query data and discrete regions of memory, every single context switch effectively evicts the L1 and L2 caches. The CPU spends hundreds of microseconds paralyzed, waiting on massive data fetches from main memory[26].
3. **Lock Contention:** PostgreSQL relies extensively on user-space spinlocks (such as buffer_strategy_lock) to protect highly contended shared memory regions like the global buffer cache[7]. The high frequency of preemption mandated by 500 competing processes drastically increases the probability of a backend being preempted while holding a critical spinlock, initiating the precise PREEMPT_LAZY lock convoying death-spiral detailed previously[8].

# Synthesizing Capacity: Active PostgreSQL Connections per CPU Core

By synthesizing the rigorous mechanics of the EEVDF scheduler, the measured latencies of context switch overheads, and PostgreSQL's rigid process architecture, it is possible to mathematically deduce the absolute physical limit of concurrently active PostgreSQL connections a single CPU core can sustain before throughput collapses.
It is vital to distinguish between total *open* connections and *active executing* connections[35]. An operating system can easily maintain thousands of idle TCP connections, provided there is sufficient RAM to absorb the 10MB-20MB per-process penalty[34]. The EEVDF scheduler only exerts pressure on the CPU when a task is runnable and inserted into the RB-tree[3].

## Mathematical Modeling of the Scheduling Epoch

To achieve optimal database throughput, the CPU must spend the vast majority of its time actively executing PostgreSQL binary instructions. Any time spent executing kernel context switches is mathematically defined as wasted overhead[24].
By utilizing the standard kernel parameters established previously, we can model the capacity of a single core:

- **EEVDF Base Slice (sysctl_sched_base_slice):** $0.75 \text{ ms}$ ($750 \ \mu\text{s}$)[10].
- **Direct Context Switch Cost:** $1.5 \ \mu\text{s}$ [25].
- **Indirect Context Switch Cost (Cache/TLB Eviction):** Conservatively $50 \ \mu\text{s}$ for a highly randomized database workload[27].
- **Total Estimated Overhead per Switch:** $\approx 51.5 \ \mu\text{s}$.

If an administrator allows **10 active connections** to simultaneously compete for a single CPU core:

- The EEVDF scheduler will assign each process a deadline based on a $750 \ \mu\text{s}$ quantum.
- Over one complete rotation of the runqueue, the core will dedicate $10 \times 750 \ \mu\text{s} = 7.5 \text{ ms}$ of wall-clock time to application execution.
- The total context switch overhead incurred per rotation is $10 \times 51.5 \ \mu\text{s} = 515 \ \mu\text{s}$ ($0.515 \text{ ms}$).
- The CPU is squandering roughly **6.8%** of its absolute physical capacity purely on the mechanical overhead of switching tasks. Furthermore, the minimum latency for any individual query is artificially inflated to $7.5 \text{ ms}$ simply because it must idle in the runqueue while the other 9 processes consume their slices[10].

If the concurrency is scaled to **100 active connections** per single CPU core:

- The total context switch overhead scales linearly: $100 \times 51.5\ \mu\text{s} = 5,150\ \mu\text{s}$ ($5.15\ \text{ms}$) wasted per scheduling rotation.
- The CPU cache is completely destroyed. Worse, the statistical probability of preempting a process while it holds an internal PostgreSQL buffer spinlock approaches certainty, triggering catastrophic lock convoying under the PREEMPT_LAZY model and bringing total throughput to a halt[8].

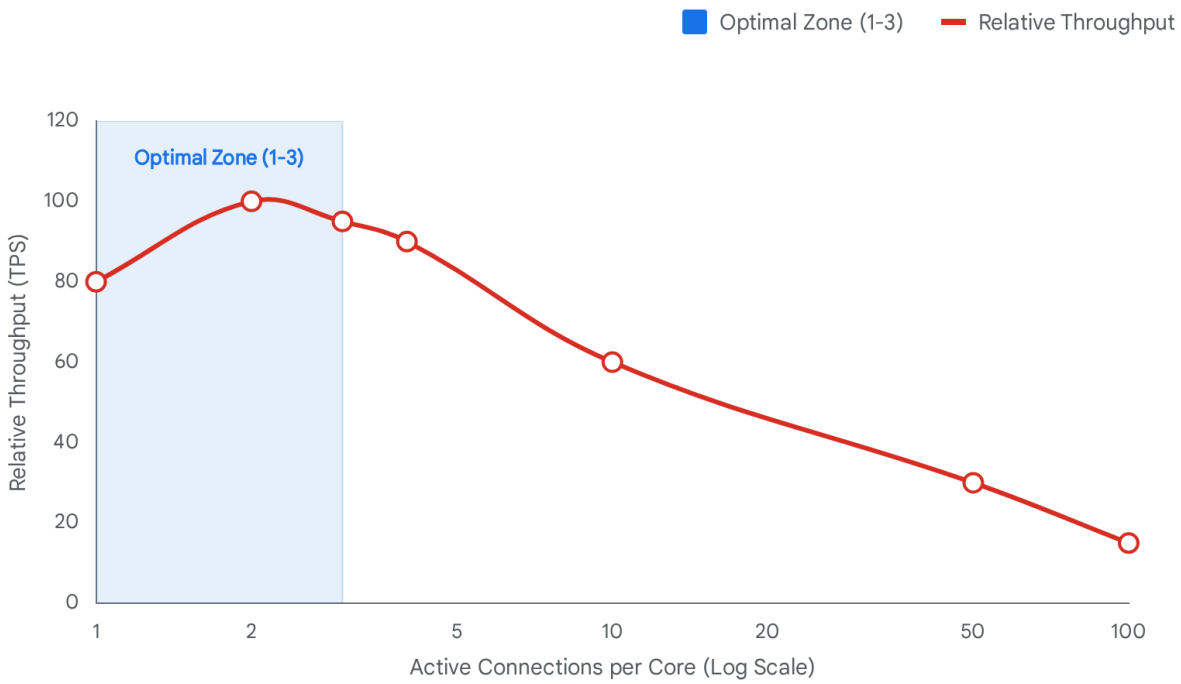
## The Optimal Concurrency Bound

Extensive industry benchmarking and architectural analysis converge on a proven mathematical formula for deriving the optimal number of active database connections across an entire server:

$$\text{Total Active Connections} \approx (\text{Core Count} \times 2) + \text{Effective Spindle Count}$$

Assuming deployment on modern NVMe flash storage architectures (where mechanical disk spindle count is irrelevant and I/O latency is negligible), the optimization formula simplifies strictly to the limitations of the CPU layer[33].
Therefore, on **a single CPU core**, the optimal number of active, concurrently executing PostgreSQL connections is **strictly constrained to a range between 1 and 3**[40].

## Throughput Collapse: The Cost of Unmanaged Concurrency




As active connections exceed the optimal threshold of 2-3 per core, the EEVDF scheduler is forced into high-frequency context switching. CPU cycles are cannibalized by cache thrashing and lock convoying, resulting in a precipitous drop in overall query throughput.

Data sources: Netdata, Tinybird, PostgreSQL Wiki, dev.to

To successfully enforce this physical hardware limit while simultaneously serving thousands of concurrent web clients, infrastructure architecture mandates the deployment of a middle-layer connection pooler[34].

## Connection Pooling Topologies

Tools such as PgBouncer or Odyssey serve as critical shields, isolating the database from the chaos of client connectivity[34]. These poolers establish a massive array of lightweight frontend connections facing the client applications, utilizing high-performance event loop mechanisms like Linux epoll to monitor thousands of idle sockets without burning CPU cycles[42]. Simultaneously, the pooler maintains a highly constrained micro-pool of persistent backend connections to the actual PostgreSQL server, sized exactly to match the hardware's core count[34].

| Pooling Mode | Architectural Behavior | Use Case Suitability |
| --- | --- | --- |
| **Session Pooling** | Maps one client connection to one backend connection for the entire duration of the client session[33]. | Least efficient. High compatibility, but fails to prevent throughput collapse under high concurrency[34]. |
| **Transaction Pooling** | Borrows a backend connection only for the duration of a specific BEGIN/COMMIT transaction block, returning it immediately to the pool[33]. | The industry standard. Highly effective for web applications, maximizing CPU throughput by keeping active processes within the 1-3 per core limit[34]. |
| **Statement Pooling** | Borrows a connection for a single statement execution. Breaks multi-statement transactions[33]. | Extreme performance for specific auto-commit workloads, but highly restrictive[43]. |

By deploying Transaction mode pooling, incoming client queries are rapidly multiplexed across a handful of active backends. This entirely circumvents the EEVDF scheduler's necessity to execute context switches, prevents cache eviction, drastically mitigates the risk of PREEMPT_LAZY lock convoying, and permits the CPU to process database instructions with maximum theoretical efficiency[34].

# Conclusion

The ongoing evolution of the Linux kernel's CPU scheduling architecture, culminating in the deployment of the Earliest Eligible Virtual Deadline First (EEVDF) scheduler, demonstrates a sophisticated maturation in operating system resource management. By abandoning heuristics in favor of strict mathematical eligibility filtering and virtual deadline prioritization, EEVDF achieves deterministic, bounded tail latencies without sacrificing the proportional equity established by its predecessor, the Completely Fair Scheduler (CFS). The profound structural elegance of the augmented Red-Black tree—which is explicitly sorted by virtual deadline while simultaneously maintaining a min_vruntime heap—allows the kernel to identify optimal tasks in $O(1)$ time, guaranteeing that the complex scheduling logic respects strict computational bounds.

However, operating system scheduling does not occur within a theoretical vacuum. The dynamic interplay between EEVDF's customizable time slices, the indirect cache-destruction costs of context switching, and the aggressive PREEMPT_LAZY preemption model dictates the operational viability of high-throughput applications. For legacy systems like PostgreSQL, which are inherently bound to rigid process-per-connection architectures and highly

vulnerable user-space spinlocks, the OS scheduler imposes an impenetrable physical boundary. The mathematical realities of context switch overhead and preemption-induced lock convoying unequivocally demonstrate that a single CPU core cannot efficiently process more than 1 to 3 concurrently active database connections within a scheduling epoch. Total mastery of database performance at scale, therefore, requires neutralizing the kernel scheduler's load entirely by routing traffic through external transaction poolers, ensuring the processor dedicates its time slices to executing complex query logic rather than mechanically swapping memory tables.

**Works cited**


1. Scheduler Evolution - Linux Kernel Internals, https://kernel-internals.org/sched/scheduler-evolution/
2. Completely Fair Scheduler - Grokipedia, https://grokipedia.com/page/Completely_Fair_Scheduler
3. CFS Scheduler — The Linux Kernel documentation, https://www.kernel.org/doc/html//v6.12/scheduler/sched-design-CFS.html
4. Introductory notes on the new Linux scheduler (EEVDF) | VZ - Vittorio Zaccaria, https://www.vittoriozaccaria.net/blog/notes-on-linux-eevdf/
5. Earliest eligible virtual deadline first scheduling - Grokipedia, https://grokipedia.com/page/Earliest_eligible_virtual_deadline_first_scheduling
6. EEVDF Scheduler - The Linux Kernel documentation, https://docs.kernel.org/scheduler/sched-eevdf.html
7. The 7.0 scheduler regression that wasn't - LWN.net, https://lwn.net/Articles/1067029/
8. aws-graviton-getting-started/linux_kernel.md at main - GitHub, https://github.com/aws/aws-graviton-getting-started/blob/main/linux_kernel.md
9. PREEMPT_NONE Is Dead; Your Postgres Probably Doesn't Care - The Build, https://thebuild.com/blog/preempt_none-is-dead-your-postgres-probably-doesnt-care/
10. Multitasking functionality - Wikibooks, open books for an open world, https://en.wikibooks.org/wiki/The_Linux_Kernel/Multitasking
11. Contention-resilient overcommitment for serverless deployments - University of Cambridge, https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-1004.pdf
12. kernel/sched/fair.c - kernel/common - Git at Google - Android GoogleSource, https://android.googlesource.com/kernel/common/+/21ed84930c160ed721c131a67e5ae6181ac40e1e/kernel/sched/fair.c
13. overview continued CFS, BFS, Deadline, MuQSS, etc. What's new in process scheduling? - students, https://students.mimuw.edu.pl/ZSO/Wyklady/14_CPUschedulers2/ProcessScheduling2.pdf
14. Earliest Eligible Virtual Deadline First (EEVDF) Scheduling in Linux - TU Dresden, https://tu-dresden.de/ing/informatik/sya/professur-fuer-betriebssysteme/die-professur/termine/echtzeit-ag/earliest-eligible-virtual-deadline-first-eevdf-scheduling-in-linux

15. An EEVDF CPU scheduler for Linux - LWN.net, https://lwn.net/Articles/925371/
16. 不懂EEVDF 调度器，别再说你懂Linux 内核了 - 51CTO, https://www.51cto.com/article/839855.html
17. [PATCH OLK-6.6 v1 0/3] optimize eevdf scheduler - Kernel - mailweb.openeuler.org, https://mailweb.openeuler.org/archives/list/kernel@openeuler.org/thread/5UCKPSHK4EETBJCU6KLKHNUC62O5ECMY/
18. Completing the EEVDF scheduler - LWN.net, https://lwn.net/Articles/969062/
19. [OLK-6.6 1/3] sched/eevdf: Sort the rbtree by virtual deadline - Kernel - mailweb.openeuler.org, https://mailweb.openeuler.org/archives/list/kernel@openeuler.org/message/KJ7UVDCOBVLDCPTIW6WEMMZ7U4LT3JBD/
20. [v1] sched: EEVDF using latency-nice | Patchew, https://patchew.org/linux/20230306132521.968182689@infradead.org/20230306141502.810909205@infradead.org/
21. Chapter 16. Kernel | Considerations in adopting RHEL 10 | Red Hat Enterprise Linux, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/considerations_in_adopting_rhel_10/kernel
22. [PATCH 22/24] sched/eevdf: Use sched_attr::sched_runtime to set request/slice suggestion, https://patchew.org/linux/20240727102732.960974693@infradead.org/20240727105030.842834421@infradead.org/
23. Ze Gao: Re: [RFC PATCH] sched/eevdf: Use tunable knob sysctl_sched_base_slice as explicit time quanta - LKML, https://lkml.org/lkml/2024/2/7/164
24. Optimizing Context Switches in the Linux Kernel: Hidden Performance Leverage | by Bervice, https://medium.com/@bervice/optimizing-context-switches-in-the-linux-kernel-hidden-performance-leverage-62feb28c9fd1
25. Measuring Context Switching and Memory Overheads for Linux Threads: A Deep Dive into NPTL Performance - Evgenii Orlov, https://eorlov.org/posts/2023/measuring-context-switching-and-memory-overheads-for-linux-threads/
26. Measuring context switching and memory overheads for Linux threads, https://eli.thegreenplace.net/2018/measuring-context-switching-and-memory-overheads-for-linux-threads/
27. Quantifying The Cost of Context Switch* - USENIX, https://www.usenix.org/events/expcs07/papers/2-li.pdf
28. Fine-Grained Computation Offload for Off-the-Shelf Servers in Tens of Lines - arXiv, https://arxiv.org/html/2607.02630v1
29. SUSE Linux Enterprise Server Release Notes, https://susedoc.github.io/release-notes/sles-16.1/html/release-notes/
30. Diff - 36ae1c45b2cede43ab2fc679b450060bbf119f1b^1..36ae1c45b2cede43ab2fc679b450060bbf119f1b - linux/kernel/git/torvalds/linux - Git at Google,

https://linux.googlesource.com/linux/kernel/git/torvalds/linux/+/36ae1c45b2cede43ab2fc679b450060bbf119f1b%5E1..36ae1c45b2cede43ab2fc679b450060bbf119f1b/
31. rseq: Implement time slice extension mechanism - LWN.net, https://lwn.net/Articles/1037202/
32. arch/m68k/kernel/syscalls · master · Ture Bentzin / linux - beim GitLab der FH Aachen!, https://git.fh-aachen.de/tb3838s/linux/-/tree/master/arch/m68k/kernel/syscalls?ref_type=heads
33. 7 Ways to Scale PostgreSQL in 2026 (When Each One Breaks) - VeloDB, https://www.velodb.io/glossary/ways-to-scale-postgresql
34. PostgreSQL Performance Optimization: Why Connection Pooling Is Critical at Scale, https://dev.to/abdullahmubin/postgresql-performance-optimization-why-connection-pooling-is-critical-at-scale-27bk
35. PostgreSQL 17 Database Administration: Mastering max_connections and Connection Management | by Jeyaram Ayyalusamy | Medium, https://medium.com/@jramcloud1/postgresql-17-database-administration-mastering-max-connections-and-connection-management-a8c28db60aad
36. Outgrowing Postgres: Handling increased user concurrency - Tinybird, https://www.tinybird.co/blog/outgrowing-postgres-handling-increased-user-concurrency
37. PostgreSQL FATAL: too many connections - causes and fixes - Netdata, https://www.netdata.cloud/guides/postgres/postgres-too-many-connections/
38. How to understand the PostgreSQL discrepancy between connection pool recommendation ((2 * n_cores) + n_disks) and support for 100s of connections? - Database Administrators Stack Exchange, https://dba.stackexchange.com/questions/82558/how-to-understand-the-postgresql-discrepancy-between-connection-pool-recommendat
39. High CPU, possibly due to context switching? - Stack Overflow, https://stackoverflow.com/questions/9535135/high-cpu-possibly-due-to-context-switching
40. Number Of Database Connections - PostgreSQL wiki, https://wiki.postgresql.org/wiki/Number_Of_Database_Connections
41. What is the relationship between number of CPU cores and number of threads in an app in java? - Stack Overflow, https://stackoverflow.com/questions/21158165/what-is-the-relationship-between-number-of-cpu-cores-and-number-of-threads-in-an
42. epoll & I/O Multiplexing — Linux | CrackingWalnuts, https://crackingwalnuts.com/linux/epoll
43. Scaling PostgreSQL Connections in Azure: A Deep Dive into Multi-PgBouncer Architectures, https://techcommunity.microsoft.com/blog/microsoftmissioncriticalblog/scaling-postgresql-connections-in-azure-a-deep-dive-into-multi-pgbouncer-archite/4420637